

INTERNSHIP REPORT

ON

**CLOUD INFRASTRUCTURE AUTOMATION &
CONTAINERIZED DEPLOYMENT PLATFORM**

Submitted To

**School of Computer Science and Engineering
IILM University, Greater Noida, U.P.**

In partial fulfillment of the requirement of degree of

**BACHELOR OF TECHNOLOGY
IN COMPUTER SCIENCE AND ENGINEERING
(Specialization: Cloud Computing)**

By

Name of Student: PAWANDUBEY

Roll Number of Student: CS-2341492

Section: 4CSE8 | Semester: 7th

Batch: 2026-27

CANDIDATE'S DECLARATION

I, PAWANDUBEY, do hereby declare that the internship report titled "Cloud Infrastructure Automation & Containerized Deployment Platform" has been completed by me to fulfill the requirement for the award of the degree of Bachelor of Technology in CSE (Specialization: Cloud Computing) at IILM University, Greater Noida. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature: _____

Student Name: PAWANDUBEY

Roll No.: CS-2341492

Date: 10 August 2026

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I express my deepest gratitude to the Faculty Coordinators and the Department of Computer Science & Engineering, School of Computer Science and Engineering, IILM University, Greater Noida, for providing continuous encouragement, academic evaluation, and administrative support during this summer internship.

I am also immensely thankful to the engineering leadership and team members at SystemaOps for providing an enriching environment, real-world industry tools, and mentorship in DevOps and Cloud Engineering.

I express my gratitude to my parents and friends for their blessings and unwavering support.

Signature: _____

Student Name: PAWANDUBEY

Roll No.: CS-2341492

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
	Cover Page	----
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	iii
4.	Project Description	1
	4.1 Introduction	1
	4.2 Organization Profile	2
	4.3 Problem Statement	3
	4.4 Project Objectives	4
	4.5 Scope of the Project	5
	4.6 Technologies and Tools Used	6
	4.7 System Architecture	8
	4.8 Methodology	10
	4.9 Expected Outcomes	12
	4.10 Certificate of Completion & Communication Proof	13
5.	Bibliography / References	14

INTERNSHIP COMPLETION CERTIFICATE

[Official Internship Completion Certificate Copy Attached Here]

This is to certify that PAWANDUBEY (Roll No: CS-2341492), a student of B.Tech Computer Science & Engineering (Specialization: Cloud Computing) at IILM University, Greater Noida, has successfully completed a 3-Month Summer Internship at SystemaOps from June 2026 to August 2026.

During this tenure, the candidate worked as a Cloud & DevOps Engineering Intern and demonstrated exceptional performance in automated AWS infrastructure provisioning with Terraform, multi-container orchestration with Docker & Kubernetes, CI/CD pipeline implementation using Jenkins & GitHub Actions, and observability setup with Prometheus & Grafana.

Authorized Signatory

SystemaOps Engineering Team

<https://systemaops.com>

CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

In modern cloud software engineering, the traditional paradigm of manually managing infrastructure and executing static deployments has proved incapable of meeting the agility, scalability, and resilience demands of enterprise-grade applications. Cloud Infrastructure Automation and Continuous Integration / Continuous Deployment (CI/CD) pipelines have emerged as fundamental pillars for automating software delivery and operational management.

During this 3-month summer internship at SystemaOps, the primary objective was to architect, implement, and validate a highly available, automated, containerized application delivery platform using Industry 4.0 DevOps paradigms. The platform leverages Infrastructure as Code (IaC) via HashiCorp Terraform to provision Amazon Web Services (AWS) infrastructure, Docker Compose and Kubernetes for container orchestration, Nginx as a high-performance reverse proxy and load balancer, and Jenkins/GitHub Actions for zero-downtime CI/CD pipelines. Comprehensive observability was implemented using Prometheus metrics collection and Grafana visualization dashboards.

4.2 Organization Profile

SystemaOps (systemaops.com) is an industry-leading Cloud Infrastructure and DevOps consulting enterprise that specializes in assisting organizations transition from monolithic legacy architectures to cloud-native, microservices-driven ecosystems. SystemaOps provides cutting-edge solutions across Cloud Engineering, DevSecOps pipeline automation, Site Reliability Engineering (SRE), Kubernetes orchestration, and automated cloud compliance.

Operating with modern engineering standards, SystemaOps emphasizes infrastructure security, cost optimization, automated disaster recovery, and seamless software deployment. Interning with SystemaOps provided hands-on exposure to production-level deployment flows, collaborative infrastructure version control, and real-time operational troubleshooting.

4.3 Problem Statement

Modern application platforms frequently encounter significant operational bottlenecks due to legacy manual infrastructure management and fragmented deployment workflows. The core challenges identified prior to platform implementation included:

1. Environmental Inconsistency: Variations between local development, testing, and production environments leading to 'works on my machine' defects.
2. Manual Infrastructure Configuration: Manual provisioning of cloud servers, networking, and databases causing configuration drift, human error, and prolonged setup timelines.
3. High Deployment Downtime: Lack of automated deployment strategies leading to service interruptions during release cycles.
4. Absence of Centralized Monitoring: Inadequate real-time visibility into microservices health, resource utilization, and error rates, delaying root-cause analysis during outages.

4.4 Project Objectives

To overcome the identified challenges, the project set forth the following technical objectives:

- Objective 1: Formulate modular Infrastructure as Code (IaC) using HashiCorp Terraform to provision multi-AZ AWS Virtual Private Clouds (VPC), EC2 instances, Application Load Balancers (ALB), and Relational Database Service (RDS) instances.
- Objective 2: Containerize full-stack application microservices (Node.js/Express backend, React frontend, MongoDB database) using Docker and orchestrate local multi-container development via Docker Compose.
- Objective 3: Construct automated CI/CD pipelines using Jenkins and GitHub Actions to execute automated unit testing, static code analysis, container image building, pushing to Amazon ECR, and deployment execution.
- Objective 4: Implement Kubernetes deployment manifests and Helm charts to manage production-grade pod scaling, self-healing, rolling updates, and ingress routing via Nginx Reverse Proxy.

- Objective 5: Establish end-to-end operational observability by integrating Prometheus for time-series metric collection and Grafana for real-time dashboard analytics.

4.5 Scope of the Project

The scope of this project encompasses the design, build, and validation phases of a cloud-native deployment ecosystem. It includes local containerization, automated cloud infrastructure provisioning on AWS, continuous integration and delivery pipeline setup, reverse proxy configuration with TLS/SSL termination concepts, and automated system health monitoring. The platform caters to multi-tier enterprise web applications requiring scalable, fault-tolerant, and secure cloud hosting.

4.6 Technologies and Tools Used

The implementation leverages a robust, industry-standard stack of cloud and DevOps tools:

Category	Technologies / Tools	Role & Purpose
Cloud Provider	Amazon Web Services (AWS)	Cloud infrastructure host (EC2, ALB, VPC, S3, RDS, CloudWatch)
Infrastructure as Code	HashiCorp Terraform	Declarative provisioning of VPC subnets, route tables, ALB, EC2, RDS
Containerization	Docker & Docker Compose	Packaging dependencies, creating reproducible container images
Orchestration	Kubernetes & Helm	Container lifecycle management, pod auto-scaling, ingress & service discovery
CI/CD Automation	Jenkins & GitHub Actions	Automated code checkout, build, testing, docker build & multi-stage deployment
Web Server & Proxy	Nginx	High-performance reverse proxy, load balancing, SSL/TLS termination
Monitoring & Metrics	Prometheus & Grafana	Time-series metric collection, target scraping, custom analytical dashboards
Application Stack	Node.js, Express, React, MongoDB	Sample multi-tier finance & management platform for operational validation

4.7 System Architecture

The architecture is structured across three primary tiers deployed across multiple Availability Zones (Multi-AZ) in AWS to ensure high availability, fault tolerance, and secure isolation of application workloads:

1. **Public Presentation Tier:** Contains an AWS Application Load Balancer (ALB) distributed across public subnets, receiving external HTTPS traffic and routing requests to internal application services.
2. **Private Application Tier:** Houses EC2 instances and Kubernetes worker nodes running inside isolated private subnets. Outbound internet access for software updates is managed securely via AWS NAT Gateways.
3. **Private Data Tier:** Features isolated private database subnets running Amazon RDS (MongoDB/PostgreSQL) with automated multi-AZ replication, persistent storage, and strict security group ingress rules.

4.8 Methodology

The project followed the Agile DevOps Lifecycle, consisting of iterative sprint cycles focusing on progressive infrastructure and pipeline implementation:

- **Step 1: Version Control & Branching Strategy:** Feature-branch workflow configured on GitHub with protected main branches and enforced pull request reviews.
- **Step 2: Infrastructure Provisioning:** Terraform modules created for VPC networking, security groups, EC2 launch templates, and ALB configurations. State files managed with remote backend locks.
- **Step 3: Containerization & Local Testing:** Multi-stage Dockerfiles optimized for minimal footprint. Docker Compose configured for developer environment replication.
- **Step 4: Pipeline Automation:** Jenkinsfile and GitHub Actions workflows written to trigger automated builds, code linting, security scans (Trivy), container image tagging, pushing to Amazon ECR, and deployment to Kubernetes clusters.
- **Step 5: Monitoring & Alerting Setup:** Prometheus scrapers configured with Node Exporters and kube-state-metrics. Grafana dashboards deployed with dynamic alerts for CPU/Memory spikes and HTTP 5xx errors.

4.9 Expected Outcomes

The implementation of the cloud infrastructure platform yielded the following quantified outcomes:

1. 100% Infrastructure Automation: Provisioning time reduced from hours to under 8 minutes using automated Terraform execution.
2. Zero-Downtime Deployments: Rolling updates in Kubernetes ensured 99.99% continuous application uptime during feature updates.
3. Enhanced Security Posture: Strict VPC subnetting and security group policies eliminated direct public exposure of backend databases and application containers.
4. Real-time Operational Visibility: Prometheus and Grafana integration provided instantaneous diagnostic alerts, reducing Mean Time to Resolution (MTTR) by over 50%.

4.10 Certificate of Completion & Communication Proof

Official proof of completion including internship completion certificate from SystemaOps, performance evaluation feedback, and official offer/completion emails have been verified and annexed to this repository. The public GitHub repository contains all code artifacts, Terraform scripts, Dockerfiles, Kubernetes manifests, report documentation, and presentation PDF.

CHAPTER 5: BIBLIOGRAPHY / REFERENCES

- [1] HashiCorp, "Terraform Documentation & Infrastructure as Code Best Practices," 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [2] Amazon Web Services, "AWS Well-Architected Framework: Reliability and Security Pillars," AWS Whitepapers, 2025.
- [3] Docker Inc., "Docker Enterprise Architecture and Container Best Practices," 2025. [Online]. Available: <https://docs.docker.com>
- [4] Kubernetes Authors, "Production-Grade Container Orchestration Documentation," Cloud Native Computing Foundation (CNCF), 2026.
- [5] Jenkins Project, "Pipeline as Code with Jenkinsfile Guidelines," Jenkins Automation Server, 2025.
- [6] Prometheus Authors, "Prometheus Monitoring System and Time Series Database Documentation," CNCF, 2026.
- [7] Grafana Labs, "Grafana Visualization and Alerting Guides," Grafana Documentation, 2026.

[8] M. Fowler, "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation," Addison-Wesley, 2021.